End-Semester Examination

Duration: 120 minutes | Total: 50 marks | 02.05.2026

Problem 1 [10 marks]

(a) Describe some of the important characteristics of cache-oblivious and cache-aware algorithms. [2 + 2]

(b) Recall the simple recursive matrix multiplication algorithm. Write a simple pseudocode for this algorithm. [2]

(c) Write a recurrence relation for the number of cache misses suffered by the algorithm. [2]

(d) Derive the asymptotic number of cache misses of this algorithm. [2]

Problem 2 [12 marks]

Answer the following questions with respect to the strongly connected components of a directed graph. Note that n is the number of vertices in the graph.

(a) Define the term strongly connected component of a directed graph with a suitable example. [2]

(b) Suppose that a directed graph has k strongly connected components. What is the maximum and minimum number of edges in the graph in terms of n and k. [2 + 2]

(c) Recall the algorithm studied in the class with respect to finding the strongly connected components of a directed graph. What kind of graphs result in the algorithm requiring $O(n^2)$ time? Justify your answer, and ensure that the graph is defined for every integer. [3]

(d) Continuing the above line of question, what kind of graphs make the algorithm work in $O(n)$ time? [3]

Problem 3 [12 marks]

Consider the 3-SAT formula over $x_1, \ldots, x_5$:

$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_4) \wedge (\neg x_2 \vee x_4) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_4 \vee x_5) \wedge (\neg x_4 \vee \neg x_5).$$

(a) Run DPLL on φ with decision order $x_1, x_2, x_3, \ldots$, always trying true first. Draw the complete search tree, marking each assignment as a *decision* or forced by *unit propagation*. [3]

(b) For the first conflict in part (a), draw the implication graph. Label each node with its literal and decision level; label each edge with the clause that caused it. Mark the conflict node κ. [4]

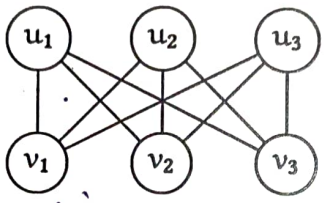
(c) Identify the last variable forced at the current decision level whose assignment directly caused the conflict in part (b). Negate the literals of all variables assigned at the current decision level that contributed to the conflict to obtain a learnt clause. State this clause and explain in one sentence why it is a logical consequence of φ. [5]

Problem 4 [16 marks]

A *dominating set* of a graph $G = (V, E)$ is a subset $S \subseteq V$ such that every vertex $v \in V \setminus S$ has at least one neighbour in S. The MINIMUM DOMINATING SET problem asks for the smallest such S.

Recall that a *vertex cover* is a subset $C \subseteq V$ such that every edge $(u, v) \in E$ has at least one endpoint in C. Note the distinction: a vertex cover must "see" every *edge*, whereas a dominating set must "reach" every *vertex* not in the set. In particular, an isolated vertex must lie in every dominating set but need not lie in any vertex cover, and a vertex cover of a connected graph is always a dominating set but the converse fails in general.

(a) **(Warm-up 1.)** Determine the treewidth of $K_{3,3}$.



[2]

(b) **(Warm-up 2.)** Consider the path P_4 on vertices $\{1, 2, 3, 4\}$ with edges $\{(1, 2), (2, 3), (3, 4)\}$.

(i) Find a minimum dominating set and a minimum vertex cover of P_4. Are they the same set? [1]

(ii) A student claims: "every minimum vertex cover is also a dominating set." Prove or disprove. [1]

(c) **(Dynamic Programming on Trees.)** Let $T = (V, E)$ be a tree rooted at r. For each vertex v, let T_v denote the subtree rooted at v. We track three states for each vertex v:

- dom[v]: minimum vertices from T_v in S, subject to $v \in S$.
- cov[v]: minimum vertices from T_v in S, subject to $v \notin S$ and at least one child of v is in S.
- free[v]: minimum vertices from T_v in S, subject to $v \notin S$ and no child of v is in S (so v must be dominated by its parent).

(i) Write the base cases for each of the three states when v is a leaf. [1]

(ii) Write the recurrences for dom[v], cov[v], and free[v] in terms of the states of v's children. Be precise about how to handle the constraint in cov[v] that *at least one* child must be in S. [3]

(d) **(Graphs of Treewidth k.)** Let $G = (V, E)$ be a graph given with a *nice* tree decomposition $\mathcal{T} = (T, \{X_t\}_{t \in V(T)})$ of width k, so $|X_t| \leqslant k + 1$ for all $t \in V(T)$. Recall that a nice tree decomposition has four node types: *leaf* ($X_t = \emptyset$), *introduce* (adds one vertex), *forget* (removes one vertex), and *join* (two children with identical bags).

For each node t and each labelling $\sigma : X_t \to \{\text{in}, \text{cov}, \text{free}\}$, define $dp[t][\sigma]$ as the minimum number of vertices placed in S among all vertices introduced at or below t, consistent with σ on the interface X_t, where:

- in: vertex is in S,
- cov: vertex is not in S but already has a neighbour in S,
- free: vertex is not in S and has no neighbour in S yet (it must be dominated by a vertex introduced later).

(i) Write the recurrence for $dp[t][\sigma]$ at each of the four node types. For the *introduce* and *join* nodes pay careful attention to how the labels of the newly introduced vertex and the consistency of cov/free labels are updated. [3]

(ii) Explain how to read off the size of a minimum dominating set from the DP table at the root of $\mathcal{T}$. [1]

(e) (Complexity and Correctness.)

(i) Prove the correctness of your treewidth DP by induction on the structure of $\mathcal{T}$: show that $dp[t][\sigma]$ equals the minimum size of a valid partial dominating set consistent with σ, or $+\infty$ if none exists. [2.5]

(ii) Show that the total number of table entries is $O(3^{k+1} \cdot |V|)$ and that each entry is computable in time $O(3^{k+1})$, giving overall runtime $O(3^{k+1} \cdot k \cdot |V|)$. Conclude that Minimum Dominating Set is fixed-parameter tractable with respect to treewidth. [1.5]